BROWSER AUTOMATION AGENT · V2.5 MULTI-AGENT ARCHITECTURE

ПОВНА ТЕХНІЧНА ДОКУМЕНТАЦІЯ ПРОГРАМНОГО КОМПЛЕКСУ «BROWSER AUTOMATION AGENT»

Інтелектуальна мультиагентна система автоматизованого проходження та верифікації веб-тестів на базі Playwright Chromium, FastAPI та великих мовних моделей

> Комплексний інженерно-архітектурний звіт за результатами повного аудиту кодової бази, історії розробки, алгоритмів інспекції DOM, підсистеми безпеки SSRF, хмарної інфраструктури Google Cloud Run + Vercel, автоматизованого тестування (96 тестів) та стратегії міграції на локальну LLM-інфраструктуру.

7 Агентів	96 Тестів	6–9 сек	100% SSRF
МУЛЬТИАГЕНТНЕ ЯДРО	65 BACKEND + 31 WEB	СЕРЕДНІЙ ЧАС ЦИКЛУ	ЗАХИСТ DNS/IP ПЕРИМЕТРА

Назва системи	Browser Automation Agent (AI Test & Quiz Automation Platform)
Автор / Розробник	Костянтин Чорний (академічна група ФЕП-33, GitHub: `@Kokos96`)
Backend-репозиторій	`https://github.com/Kokos96/browser-agent` (гілка `main`, коміт `03b2c9a`)
Frontend-репозиторій	`https://github.com/Kokos96/browser-agent-web` (гілка `main`, коміт `7b5a632`)
Production-середовище	Frontend: `https://browser-agent-web-nine.vercel.app/` · Backend: Google Cloud Run (`europe-west1`)
Контрольні стенди	`/test-quiz.html` (3 питання, профіль) та `/agent-test.html` (1 питання, smoke-тест)
Технологічний стек	Python 3.12, FastAPI, Playwright 1.63.0 (Chromium), Pydantic v2, Google GenAI SDK, Vanilla JS ES2022
Версія та дата ревізії	Версія документації 2.5 (Фінальна ревізія) · Жовтень 2026 р.

Виконавче резюме (Executive Summary)

Browser Automation Agent — це повностекна хмарна програмна система, призначена для автономного аналізу, навігації, заповнення та верифікації інтерактивних веб-форм, освітніх квізів і контрольних стендів у справжньому безголовому браузері **Playwright Chromium** під керуванням інтелектуального мультиагентного оркестратора. На відміну від простих HTTP-скраперів або монолітних LLM-скриптів, система розділяє відповідальність між **7 спеціалізованими програмними агентами** (оркестратор `QuizOrchestrator`, детектор стану сторінки, навігаційний агент, екстрактор питань, планувальник відповідей, виконавець браузерних дій та верифікатор результатів).

У процесі еволюції проєкту були повністю усунені типові хмарні вразливості та операційні збої: впроваджено криптографічну ізоляцію історії пристроїв за заголовком `X-Client-Id` (SHA-256), реалізовано мережевий фільтр `public_url_security.py` для захисту від SSRF-атак при роботі з довільними публічними HTTPS-посиланнями, обмежено життєвий цикл підпроцесів (`300 с`) із автоматичною реконсиляцією завислих сесій (`360 с`) та налаштовано виділення процесорного часу Cloud Run у режимі `--no-cpu-throttling`. Клієнтський веб-додаток забезпечує мобільну ергономіку, 6-фазний моніторинг у реальному часі (*Live Activity*) та прозорий облік спожитих токенів моделі.

Зміст документації та реєстр візуальних матеріалів

Розділ 1. Загальний огляд системи, мета та постановка задачі

Розділ 2. Еволюція проєкту та інженерна історія розробки (PR #3–#11)

Розділ 3. Високорівнева системна архітектура та потоки взаємодії

Розділ 4. Технологічний стек та обґрунтування архітектурних рішень

Розділ 5. Детальний аналіз Frontend-системи та UI/UX (8 скриншотів)

Розділ 6. Детальний аналіз Backend-системи та специфікація REST API

Розділ 7. Мультиагентна архітектура ядра (7 спеціалізованих ролей)

Розділ 8. Алгоритми інспекції DOM, планування відповідей та кліків

Розділ 9. Моделі даних, атомарна персистентність та життєвий цикл

Розділ 10. Безпека системи, захист від SSRF та межі довіри

Розділ 11. Хмарна інфраструктура (Cloud Run + Vercel) та CI/CD

Розділ 12. Тестування (96 тестів), продуктивність та матриця ризиків

Розділ 13. Перспективи розвитку та перехід на власну локальну LLM

Додатки А–Г. Специфікація помилок, глосарій та контрольний чек-лист

Карта ілюстрацій та таблиць: Документ містить **26 пронумерованих рисунків** (18 векторних інженерних діаграм `01-system-architecture.svg` .. `18-future-roadmap-local-llm.svg` та 8 реальних скриншотів інтерфейсу користувача `Рисунок 5.1` .. `Рисунок 5.8`) і **12 структурованих таблиць**. Кожен графічний елемент розміщено безпосередньо в тому розділі, який він ілюструє.

Розділ 1. Загальний огляд системи та постановка інженерної задачі

1.1. Проблематика автоматизації сучасних інтерактивних веб-тестів

Сучасні освітні платформи, системи дистанційного контролю знань (LMS) та інтерактивні веб-опитувальники використовують динамічний рендеринг DOM, багатосторінкову пагінацію, стартові екрани з обов'язковою ідентифікацією студента, модальні оверлеї та нестандартну розмітку варіантів відповідей (кастомні `<label>`, `role="radio"`, картки з обробниками подій). Класичні засоби автоматизації на основі жорстко закодованих XPath/CSS-селекторів ламаються при найменшій зміні верстки, тоді як наївні LLM-агенти («промпт на кожен клік») страждають від високої затримки, надмірної витрати токенів, галюцинацій селекторів і хибних звітів про успішне завершення.

1.2. Ключові цілі проєкту та критерії інженерної якості

- **Достовірність результатів (Zero Fake Completions):** агент переходить у статус `completed` виключно після фактичного вибору відповідей у DOM, підтвердженого надсилання форми та зчитування фінального балу або екрана завершення.
- **Семантичне розпізнавання бар'єрів доступу:** якщо зовнішній ресурс вимагає авторизації, платної підписки (paywall) або проходження CAPTCHA, система негайно класифікує сторінку та повертає чесний діагностичний статус (`blocked_access`, `subscription_required`, `needs_manual_input`) замість нескінченного циклу.
- **Безпечна робота з публічними URL:** підтримка будь-яких зовнішніх HTTPS-тестів за умови суворої перевірки DNS та блокування внутрішніх/хмарних метаданих (SSRF Guard).
- **Прозора телеметрія та мобільна доступність:** відображення прогресу по кожному питанню, підрахунок токенів Gemini (запит/відповідь/разом) та ізоляція історії кожного клієнтського браузера.

Таблиця 1.1. Класифікація функціональних підсистем за фактичним статусом реалізації

Підсистема / Модуль	Ключові файли реалізації	Статус	Доказ верифікації
Мультиагентний оркестратор (7 агентів)	`src/browser_agent/agents/*.py`	**РЕАЛІЗОВАНО** **ПЕРЕВІРЕНО**	Коміт `03b2c9a`, `test_multi_agent_orchestrator.py`
Захист від SSRF для публічних HTTPS URL	`public_url_security.py`, `api_server.py`	**РЕАЛІЗОВАНО** **ПЕРЕВІРЕНО**	Коміт `4c081b6`, `test_public_url_security.py`
Контроль таймаутів (300с) та Stale-сесій (360с)	`api_server.py`, `agent.py`	**РЕАЛІЗОВАНО** **РОЗГОРНУТО**	Коміт `555aa1b`, `test_run_timeout.py`
Ізоляція історії за `X-Client-Id` (SHA-256)	`api_server.py`, `script.js`	**РЕАЛІЗОВАНО** **РОЗГОРНУТО**	Коміт `46e0ba6`, `test_history_isolation.py`
Мобільний Web SPA V2.5 + 2 тестові стенди	`index.html`, `script.js`, `agent-test.html`	**РЕАЛІЗОВАНО** **РОЗГОРНУТО**	Коміт `7b5a632`, Vercel Production, 31 автотест
Локальний LLM-кластер (vLLM / Ollama / LoRA)	Архітектурний інтерфейс `llm.py`	**ЗАПЛАНОВАНО (V3.0)**	Детальний проєкт міграції у Розділі 13

Розділ 2. Еволюція проєкту та інженерна історія розробки

Історія репозиторіїв `Kokos96/browser-agent` та `Kokos96/browser-agent-web` демонструє послідовну трансформацію дослідницького скрипта у відмовостійку продуктивну систему через п’ять інженерних ітерацій (див. **Рисунок 2.1**).



Рисунок 2.1. Хронологія еволюції Browser Automation Agent від монолітного прототипу до мультиагентної системи v2.5

Таблиця 2.1. Реєстр ключових інженерних виправлень та архітектурних PR

Етап / PR / Коміт	Виявлена інженерна проблема (Root Cause)	Впроваджене технічне рішення
Backend PR #5–#6 `2559bc1`, `555aa1b`	Зависання запуску `5471ec26...` у статусі «Виконується» на 56+ с через фоновий тротлінг CPU у Cloud Run та відсутність верхньої межі часу роботи підпроцесу.	Додано жорсткий таймаут `MAX_RUN_DURATION_SECONDS = 300`, фонову реконсиляцію `reconcile_stale_runs()` після `360 с`, атомарний запис `os.replace()` та прапорець Cloud Run `--no-cpu-throttling`.
Frontend PR #3–#5 `20d20fa` .. `1fa2d62`	У мобільному модальному вікні логів відображався плейсхолдер або `[object Object]`, а нижній тулбар перекривав кнопку закриття звіту.	Реалізовано нормалізацію об’єктних логів `formatLogEntry()`, збереження повного логу у `results/run_<id>.log` та адаптивні відступи мобільного діалогу.
Backend PR #7 `4c081b6`	Жорсткий білий список хостів блокував реальні зовнішні тести користувачів, але відкриття довільних URL створювало ризик SSRF-атак на внутрішню мережу хмари.	Створено модуль `public_url_security.py`: дозвіл на будь-який публічний HTTPS-хост із попередньою перевіркою DNS-відповідей та блокуванням RFC1918, loopback і `169.254.169.254`.
Backend PR #8–#11 `506f807` .. `03b2c9a` **Frontend PR #7–#8**	Монолітний агент не розпізнавав платні підписки (paywall), повторно клікав уже вибрані радіокнопки та міг завершити сесію з результатом `0/0`.	Повний редизайн у 7-агентну систему (`QuizOrchestrator`), пакетне планування відповідей, фільтрація відповідених питань, детекція `SUBSCRIPTION_REQUIRED` та новий стенд `agent-test.html` (`7b5a632`).

Розділ 3. Високорівнева системна архітектура та потоки взаємодії

3.1. Трирівнева топологія комплексу

Архітектура **Browser Automation Agent** побудована за принципом чіткого розділення презентаційного рівня, рівня оркестрації/виконання та зовнішніх інфраструктурних сервісів (див. **Рисунок 3.1**):

1. **Клієнтський презентаційний рівень (Vercel Edge Network):** односторінковий веб-додаток (SPA), написаний на чистому HTML5, CSS3 та Vanilla JavaScript без важких бандлерів. Відповідає за конфігурацію профілю студента, передачу параметрів запуску, адаптивне опитування стану сесії та візуалізацію аналітики. Разом із фронтендом на Vercel розміщено два еталонні автономні стенди: `/test-quiz.html` та `/agent-test.html`.
2. **Серверний рівень керування та браузерної автоматизації (Google Cloud Run, `europe-west1`):** контейнеризований сервіс на базі FastAPI (`api_server.py`), який виконує криптографічну перевірку ключа `X-API-Key`, SSRF-валідацію цільового URL та породжує ізольований асинхронний підпроцес Python (`main.py`). Усередині підпроцесу працює мультиагентне ядро та екземпляр Headless Chromium під керуванням Playwright.
3. **Рівень когнітивного виведення (Google Gemini API):** зовнішній хмарний сервіс генеративного ШІ, підключений через офіційний SDK `google-genai`. Використовується агентом `AnswerPlanner` для пакетного вибору правильних відповідей у форматі строго типізованого JSON та як резервний класифікатор складних станів сторінки.

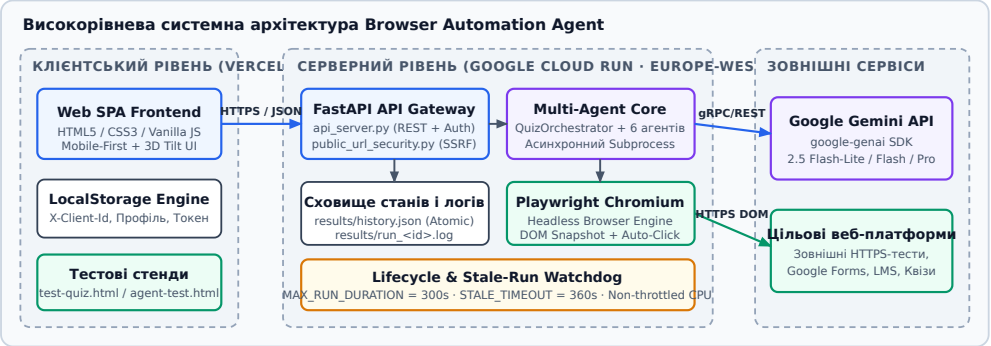


Рисунок 3.1. Високорівнева системна архітектура програмного комплексу Browser Automation Agent

3.2. Протокол комунікації між Frontend SPA та FastAPI Backend

Оскільки повний цикл проходження багатосторінкового тесту у справжньому браузері може тривати від 5 до 60+ секунд, синхронна модель «один HTTP-запит — фінальна відповідь» є неприйнятною через тайм-аути проксі-серверів і відсутність зворотного зв’язку для користувача. Тому реалізовано **асинхронну неблокуючу схему з адаптивним поллінгом** (див. **Рисунок 3.2**).

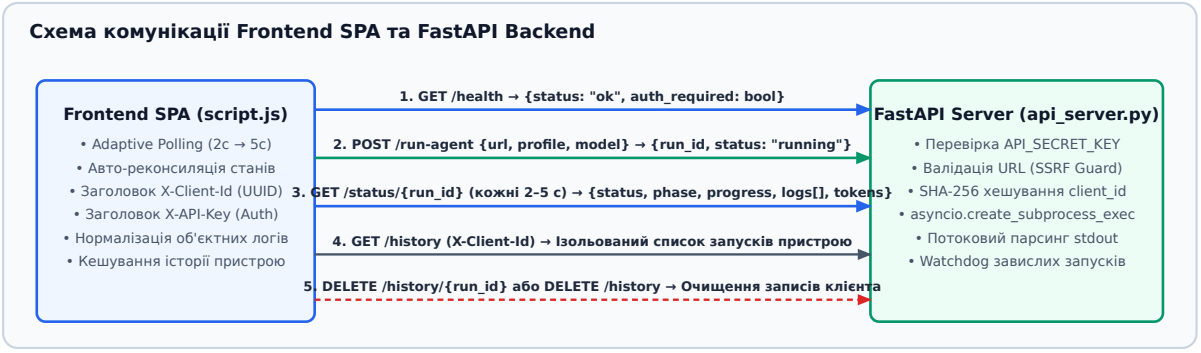


Рисунок 3.2. Протокол взаємодії між клієнтським додатком (Vercel) та серверним API (Google Cloud Run)

Після виклику `POST /run-agent` сервер негайно (за `< 80 мс`) повертає згенерований 32-символьний ідентифікатор `run_id` та початковий статус `running`. Клієнтський скрипт запускає цикл опитування `GET /status/{run_id}` з інтервалом **2 секунди** протягом перших 15 секунд активної фази, після чого автоматично збільшує інтервал до **5 секунд** для мінімізації мережевого навантаження. Кожен запит супроводжується заголовком `X-Client-Id` (унікальний UUIDv4 пристрою з `localStorage`), який на бекенді хешується за алгоритмом `SHA-256` для гарантованої ізоляції історії запусків між різними користувачами.

Розділ 4. Технологічний стек та обґрунтування архітектурних рішень

Вибір технологічних компонентів підпорядкований вимогам мінімального часу холодного старту контейнера в Cloud Run, детермінованості парсингу DOM та відсутності зайвих залежностей на клієнті (див. **Таблицю 4.1**).

Таблиця 4.1. Специфікація технологічного стека та обґрунтування інженерного вибору

Рівень	Технологія / Пакет	Версія	Інженерне призначення та переваги вибору
Базовий образ	`mcr.microsoft.com/playwright/python`	`v1.63.0-noble`	Офіційний образ Microsoft на базі Ubuntu 24.04 (Noble) з попередньо встановленими системними бібліотеками Chromium, шрифтами та драйверами.
Мова Backend	Python	`3.12`	Висока швидкість роботи `asyncio`, типізація `typing` та нативна інтеграція з AI-екосистемою.
API-фреймворк	FastAPI + Uvicorn (standard)	`latest`	Асинхронна обробка конкурентних HTTP-запитів, автоматична валідація корисного навантаження та мінімальні накладні витрати.
Браузерний рушій	Playwright (Python Sync/Async)	`1.63.0`	Повна підтримка сучасного JS/CSS, автоматичне очікування стабільності елементів (auto-wait), перехоплення навігації та мультиконтекстна ізоляція.
LLM SDK	`google-genai`	`latest`	Офіційний SDK нового покоління для моделей сімейства `gemini-2.5-flash-lite`, `gemini-2.5-flash` та `gemini-2.5-pro` з підтримкою метаданих токенів.
Валідація даних	Pydantic v2	`>=2.0`	Строга перевірка схем запитів API, станів оркестратора (`WorkflowState`), екстрагованих питань і JSON-відповідей від LLM.
Frontend SPA	HTML5 + CSS3 + Vanilla JS	`ES2022`	Нульовий час збірки, миттєве завантаження на мобільних мережах, кастомна 3D-перспектива карток та адаптивна верстка під iOS/Android.
Тестовий стек	`pytest` / `unittest` + `@playwright/test`	`Python 3.12 / Node 20`	65 модульних/інтеграційних тестів бекенду + 28 регресійних перевірок JS + 3 наскрізні браузерні E2E-сценарії.

Розділ 5. Детальний аналіз Frontend-системи та користувацького інтерфейсу

5.1. Архітектура клієнтського додатка та Головний екран («Головна»)

Фронтенд-репозиторій `Kokos96/browser-agent-web` реалізує мобільно-орієнтований односторінковий інтерфейс у стилі висококонтрастної темної теми (*Dark Cyber-Academic UI*: фон `#08090c`, поверхні карток `#11131a`, неоновий акцент `#d4ff00`). Верхня панель (Header) постійно відображає індикатор стану бекенду (`API підключено` / `Потрібен токен` / `API недоступний`), який оновлюється через `GET /health`, та інтерактивний чіп активного профілю студента (`КЧ Костянтин`).

На **Рисунку 5.1** та **Рисунку 5.2** представлено верхню та нижню частини головного екрана («Головна»). Картка запуску містить поле вводу цільового URL із автоматичним очищенням пробілів, кнопки швидкого вибору контрольованих стендів (`Контрольний стенд (E2E Stand)`), компактне зведення профілю студента (`Костянтин Чорний (ФЕП-33)` із кнопкою `Змінити`), селектор моделі Gemini та головну кнопку дії `Запустити виконання тесту`. Нижче розташована сітка з **4 KPI-карток** у 3D-перспективі (`Останній результат`, `Токени Gemini`, `Середній час`, `Всього запусків`) та прев’ю секції `Останні запуски`.

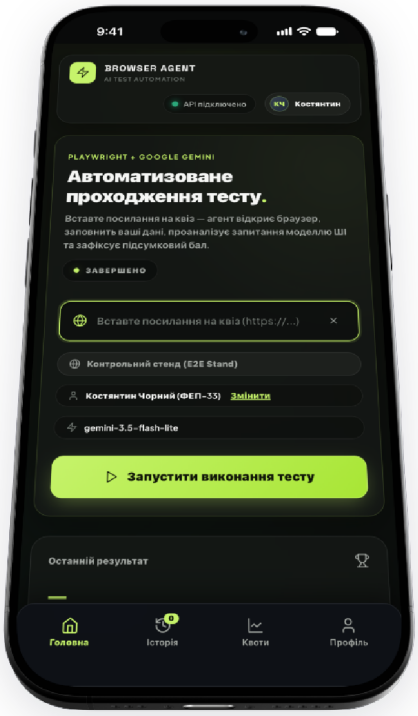


Рисунок 5.1. Головний екран («Головна», верхня частина): індикатор API, поле URL, пресет контрольного стенду, профіль студента та вибір моделі Gemini



Рисунок 5.2. Головний екран («Головна», нижня частина): 4 агреговані KPI-картки у 3D-перспективі, блок «Останні запуски» та нижня навігація

5.2. Моніторинг активного виконання у реальному часі (Live Activity) та Екран «Історія»

Одразу після натискання кнопки запуску інтерфейс переходить у режим активного відстеження (див. **Рисунок 5.3**). Головна кнопка блокується з підписом `Агент виконує тест...`, щоб унеможливити дублювання підпроцесів, і розгортається блок **«Хід виконання агента (Live Activity)»** зі статусом `ВИКОНУЄТЬСЯ`. Клієнтський парсер у `script.js` зіставляє серверні фази оркестратора з **6 наочними кроками таймлайну**: (1) *Запуск браузера*, (2) *Відкриття тесту*, (3) *Підготовка профілю*, (4) *Аналіз питань*, (5) *Вибір та надсилання відповідей*, (6) *Результат*, а також виводить поточний прогрес по питаннях (наприклад, `Питання 2/3`) та останній рядок логу.

Вкладка **«Історія запусків»** (див. **Рисунок 5.4**) надає повний журнал сесій поточного пристрою з миттєвим текстовим пошуком за URL, балом або повідомленням, фільтрацією за статусом (`Усі статуси`, `Завершено`, `Помилка`, `Виконується`) та сортуванням за датою чи тривалістю. Кожна картка історії відображає статус, дату, скорочений `Run ID`, цільовий URL, тривалість (`Час: 6 с`), фінальний бал (`Результат: 3/3`) та лічильник підтверджених відповідей (`Відповідей вибрано: 3/3`).

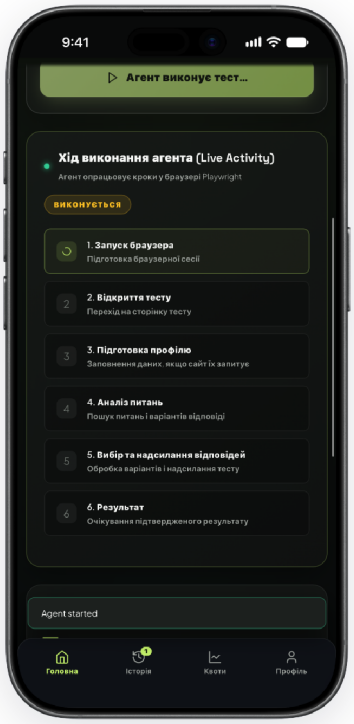


Рисунок 5.3. Активне виконання тесту (Live Activity): 6-фазний інтерактивний таймлайн прогресу Playwright та потоковий лог сеансу

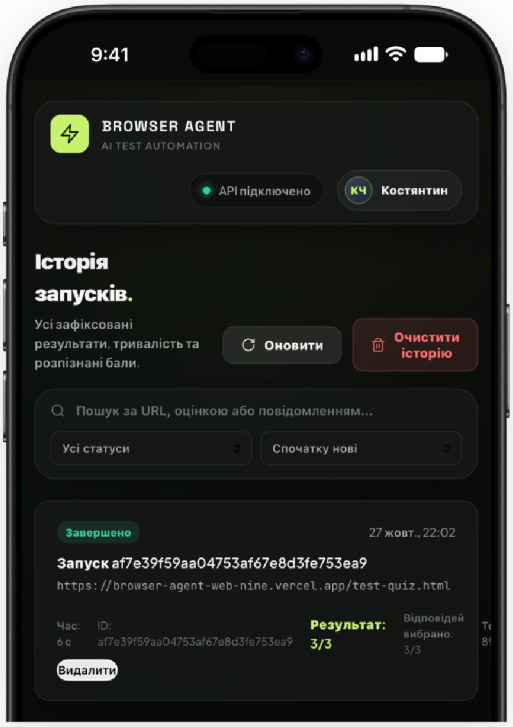


Рисунок 5.4. Екран «Історія запусків»: пошук, фільтри та картка успішного сеансу af7e39f59aa04753af67e8d3fe753ea9 (Час: 6 с, Результат: 3/3)

5.3. Модальне вікно звіту сеансу та Екран «Профіль студента»

При натисканні на будь-яку картку в історії відкривається **модальне вікно повного звіту запуску** (див. **Рисунок 5.5**, сеанс `af7e39f59aa04753af67e8d3fe753ea9`). Воно містить структуровану таблицю телеметрії: `СТАТУС: Завершено`, `ТРИВАЛІСТЬ: 6 с`, точні часові мітки початку й завершення, повний `RUN ID`, `ПИТАНЬ РОЗПІЗНАНО: 3`, `ВІДПОВІДЕЙ ВИБРАНО: 3/3`, `РЕЗУЛЬТАТ ТЕСТУ: 3/3` та детальну розбивку використаних токенів: `ТОКЕНИ GEMINI: 892 (запит: 560, відповідь: 332)`. Нижче виводяться клікабельний URL, діагностичне резюме оркестратора та прокручуваний блок повних логів сесії, нормалізованих функцією `formatLogEntry()`.

Вкладка **«Профіль студента та з’єднання»** (див. **Рисунок 5.6**) дозволяє налаштувати персональні дані (`Ім’я: Костянтин`, `Прізвище: Чорний`, `Академічна група: ФЕП-33`), які автоматично підставляються агентом у реєстраційні поля тестів, обрати модель Gemini за замовчуванням та проконтролювати з’єднання з хмарним хостом Cloud Run (`europe-west1.run.app`).

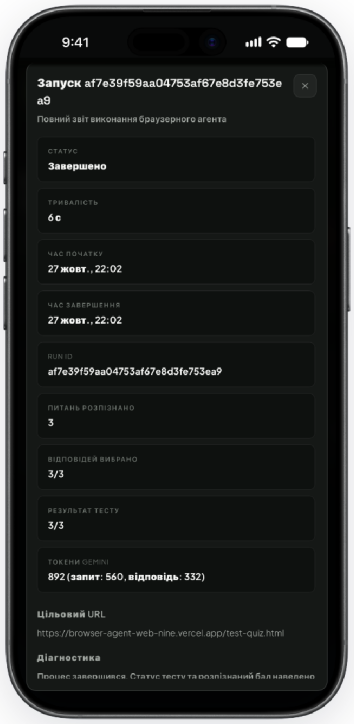


Рисунок 5.5. Модальне вікно деталей запуску af7e39f59aa04753af67e8d3fe753ea9: верифікований бал 3/3, тривалість 6 с та 892 токени Gemini

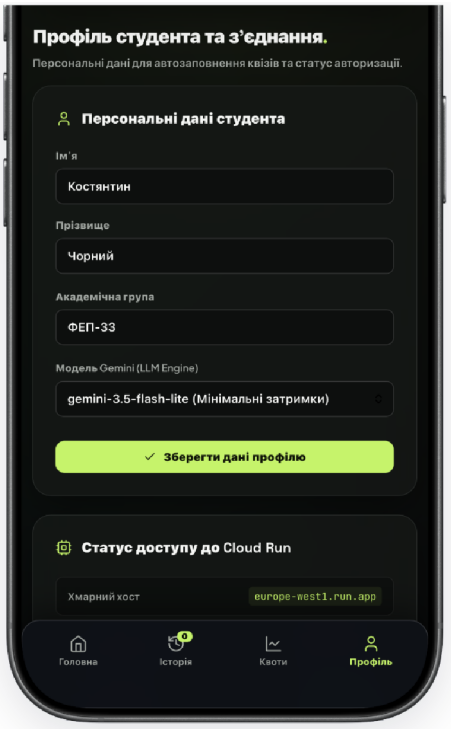


Рисунок 5.6. Екран «Профіль студента та з'єднання»: налаштування автозаповнення (Костянтин Чорний, ФЕП-33), вибір LLM та статус Cloud Run

5.4. Екран «Квоти та ШІ-аналітика»

Вкладка **«Квоти»** (див. **Рисунок 5.7** та **Рисунок 5.8**) агрегує статистику споживання ресурсів Google Gemini за всіма сесіями пристрою. Верхній блок відображає сумарну кількість використаних токенів, частку успішно завершених процесів та останній розпізнаний бал. Нижній блок будує **стовпчикову діаграму активності запусків за останні 7 днів тижня** (`ср–вт`), **кільцеву діаграму розподілу статусів** (`Завершені`, `Помилки`, `Інші`) та технічну картку конфігурації `Google Cloud Run · Gemini Inference Engine`.

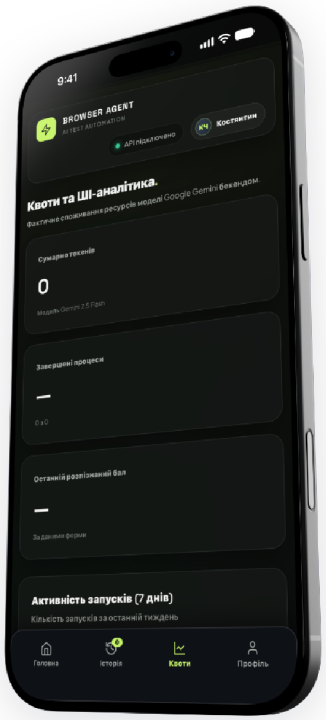


Рисунок 5.7. Екран «Квоти та ШІ-аналітика» (верхня частина): зведені KPI-метрики спожитих токенів, завершених процесів та оцінок

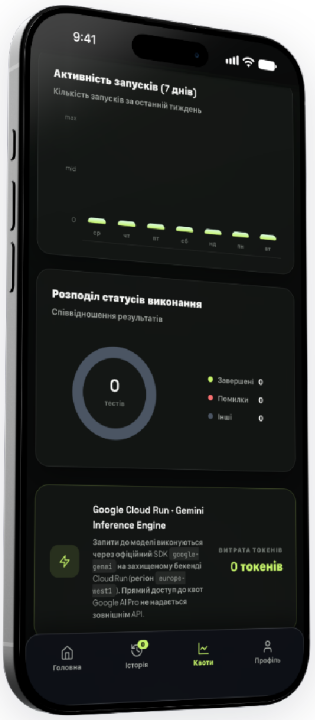


Рисунок 5.8. Екран «Квоти та ШІ-аналітика» (нижня частина): 7-денна гістограма запусків, кільцева діаграма статусів та картка Cloud Run

Розділ 6. Детальний аналіз Backend-системи та специфікація REST API

6.1. Модульна структура репозиторію `Kokos96/browser-agent`

Серверна частина організована як стандартний Python-пакет із розміщенням бізнес-логіки в каталозі `src/browser_agent/` та точками входу верхнього рівня `api_server.py` (REST-сервер FastAPI) і `main.py` (CLI-запуск агента).

```
browser-agent/
├── api_server.py                    # FastAPI шлюз, авторизація, запуск підпроцесів, реконсиляція історії
├── public_url_security.py           # SSRF-захист: валідація схеми HTTPS, DNS-резолвінг та перевірка IP-мереж
├── main.py                          # CLI-точка входу ізольованого підпроцесу агента
├── Dockerfile                       # Контейнер mcr.microsoft.com/playwright/python:v1.63.0-noble (порт 8080)
├── src/browser_agent/
│   ├── config.py                    # Конфігурація середовища, моделей Gemini та лімітів виконання
│   ├── models.py / schemas.py       # Pydantic v2 моделі станів, елементів DOM, питань, планів та звітів
│   ├── browser.py                   # Керування Playwright Chromium, інспекція DOM, автозаповнення профілю
│   ├── llm.py / prompts.py          # Інтеграція з google-genai SDK, облік токенів та системні промпти
│   ├── agent.py                     # Фасад BrowserAgent, таймаут життєвого циклу та атомарний save_history()
│   └── agents/                      # Мультиагентне ядро v2.5 (7 спеціалізованих агентів)
│       ├── orchestrator.py          # Agent A: QuizOrchestrator — координатор станів та бюджету кроків
│       ├── state_detector.py        # Agent B: PageStateDetector — класифікація 11 категорій сторінок
│       ├── navigation_agent.py      # Agent C: NavigationAgent — пошук кнопок старту та пагінації
│       ├── question_extractor.py    # Agent D: QuestionExtractor — мультистратегічний парсинг питань
│       ├── answer_planner.py        # Agent E: AnswerPlanner — пакетне планування відповідей через Gemini
│       ├── action_executor.py       # Agent F: BrowserActionExecutor — надійні кліки та обхід оверлеїв
│       └── submission_agent.py      # Agent G: SubmissionVerifier — фінальний сабміт і витягнення балів
└── tests/                           # 65 автоматизованих тестів (unit, security, lifecycle, multi-agent)
```

6.2. Повна специфікація REST API (`api_server.py`)

Усі ендпоінти (крім діагностичного `GET /health`) захищені залежністю `verify_api_key`, яка порівнює заголовок `X-API-Key` зі змінною середовища `API_SECRET_KEY` за допомогою захищеної від таймінг-атак функції `hmac.compare_digest()`.

Таблиця 6.1. Контракти REST API ендпоінтів сервера FastAPI

Метод та шлях	Заголовки / Тіло запиту	Повертає (JSON 200 OK)	Призначення та коди помилок
`GET /health`	Без авторизації	`{"status": "ok", "auth_required": bool}`	Перевірка доступності Cloud Run та режиму захисту ключем.
`POST /run-agent`	`X-API-Key`, `X-Client-Id` `{"url", "first_name", "last_name", "group", "model"}`	`{"run_id": "<32-hex>", "status": "running", "phase": "starting"}`	Валідує URL через `validate_target_url()`, створює запис історії та запускає підпроцес `main.py`. Помилки: `400` (небезпечний URL), `401/403` (ключ), `422` (схема).
`GET /status/{run_id}`	`X-API-Key`, `X-Client-Id`	`{"run_id", "status", "phase", "progress", "score", "logs", "gemini_tokens", ...}`	Повертає оперативний стан із пам’яті або з `results/history.json` (разом із вмістом `results/run_{id}.log`). Код `404`, якщо не знайдено.
`GET /history`	`X-API-Key`, `X-Client-Id`	`[HistoryEntry, ...]`	Виконує `reconcile_stale_runs()` та повертає список сесій, відфільтрований за `SHA-256(X-Client-Id)`.
`DELETE /history/{id}` `DELETE /history`	`X-API-Key`, `X-Client-Id`	`{"status": "deleted"}` або `{"deleted_count": N}`	Видаляє один або всі завершені записи історії, що належать поточному пристрою клієнта.

6.3. Асинхронний підпроцесний рушій та потоковий парсинг телеметрії

Запуск браузера Playwright в окремому процесі OS через `asyncio.create_subprocess_exec(sys.executable, "main.py", ...)` забезпечує повну ізоляцію пам’яті Chromium від основного циклу подій Uvicorn. Фонова корутина `collect_output(run_id, process)` у реальному часі зчитує `stdout` підпроцесу по рядках, записує їх у `results/run_{run_id}.log` та розпізнає структуровані маркери прогресу (фази `starting`, `opening_url`, `preparing_profile`, `analyzing_questions`, `selecting_answers`, `submitting`, лічильники питань `questions_found`, `questions_answered` і статистику токенів), миттєво оновлюючи стан для клієнтського поллінгу.

Розділ 7. Мультиагентна архітектура ядра та оркестрація виконання

7.1. Декомпозиція на 7 спеціалізованих агентських ролей

Ключовим інженерним досягненням версії v2.5 (коміт `03b2c9a`) є перехід від монолітного циклу «спостереження → один LLM-клік» до **державно-орієнтованої мультиагентної архітектури з 7 вузькоспеціалізованих ролей** (див. **Рисунок 7.1** та **Таблицю 7.1**). Кожен агент має строго визначений вхідний і вихідний Pydantic-контракт та покритий ізольованими модульними тестами.

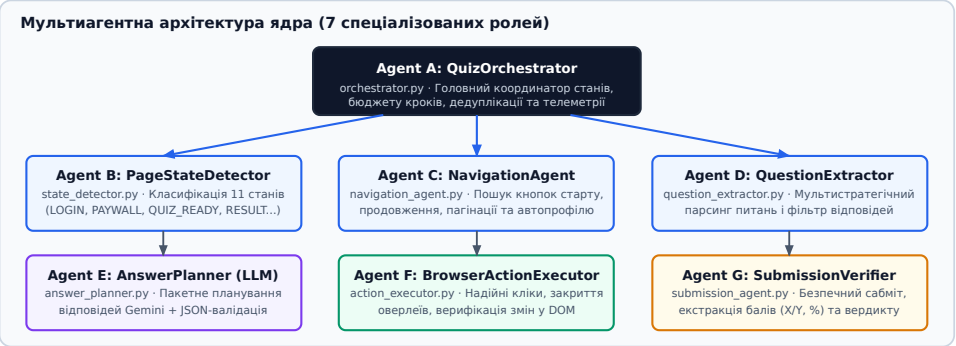


Рисунок 7.1. Структурна схема мультиагентного ядра Browser Automation Agent (7 спеціалізованих агентських ролей)

Таблиця 7.1. Матриця відповідальності та контрактів 7 агентів системи

Роль / Клас	Модуль	Вхідні дані	Вихідний контракт	Ключова функція в конвеєрі
Agent A `QuizOrchestrator`	`orchestrator.py`	`url`, `StudentProfile`, `BrowserController`, `LLMClient`	`OrchestratorSummary`	Керує скінченним автоматом `WorkflowState`, бюджетом кроків (`max_steps=25`), захистом від зациклення та фінальним вердиктом.
Agent B `PageStateDetector`	`state_detector.py`	`PageObservation` (DOM, URL, текст, форми)	`PageStateDetection` (`PageCategory` + `confidence`)	Класифікує сторінку на 11 станів: `LANDING_PAGE`, `ACTIVE_QUIZ`, `RESULT_PAGE`, `AUTH_REQUIRED`, `ACCESS_GATE` (підписка/paywall), `CAPTCHA_REQUIRED` тощо.
Agent C `NavigationAgent`	`navigation_agent.py`	`PageObservation`, `StudentProfile`	`NavigationPlan` (кнопки старту/далі, поля профілю)	Знаходить кнопки *«Почати тест»*, *«Start Quiz»*, *«Далі / Next»* та поля вводу імені, прізвища й академічної групи.
Agent D `QuestionExtractor`	`question_extractor.py`	Жива сторінка Playwright + `PageObservation`	`list[ExtractedQuestion]`	Витягує тестові питання та варіанти відповідей за 4 стратегіями, відфільтровуючи вже вибрані (відповідені) питання.
Agent E `AnswerPlanner`	`answer_planner.py`	`list[ExtractedQuestion]`, `LLMClient`	`list[PlannedAnswer]` + метадані токенів	Формує один пакетний запит до Gemini для всіх питань екрана, валідує JSON-відповідь та забезпечує детермінований резервний вибір.
Agent F `BrowserActionExecutor`	`action_executor.py`	`PlannedAnswer` або `NavigationAction`	`ActionExecutionResult` (`verified: bool`)	Виконує надійний вибір радіокнопок, чекбоксів, списків і текстових полів із автоматичним закриттям оверлеїв та перевіркою DOM.
Agent G `SubmissionVerifier`	`submission_agent.py`	`PageObservation`, лічильники відповідей	`SubmissionOutcome` (`score`, `passed`, `status`)	Натискає фінальну кнопку *«Завершити / Надіслати»* (або передає ручне керування) та витягує оцінку (`3/3`, `100%`, бали).

7.2. Наскрізний конвеєр та скінченний автомат станів (`WorkflowState`)

Послідовність взаємодії підсистем у часі та граф переходів скінченного автомата наведено на **Рисунку 7.2**, **Рисунку 7.3** та **Рисунку 7.4**. Важливою особливістю оркестратора є розмежування технічних збоїв та зовнішніх бізнес-обмежень: якщо `PageStateDetector` виявляє вимогу платної підписки або тарифного плану (маркери *«оформити підписку»*, *«upgrade to pro»*, *«платний доступ»*), сеанс негайно завершується зі статусом `subscription_required`; при виявленні форми входу

— `blocked_access`; якщо ж усі питання успішно вибрані, але кнопка фінального надсилання потребує ручного підтвердження користувача — `needs_manual_input`.



Рисунок 7.2. Наскрізний конвеєр опрацювання тестового сеансу від HTTP-запиту до збереження телеметрії

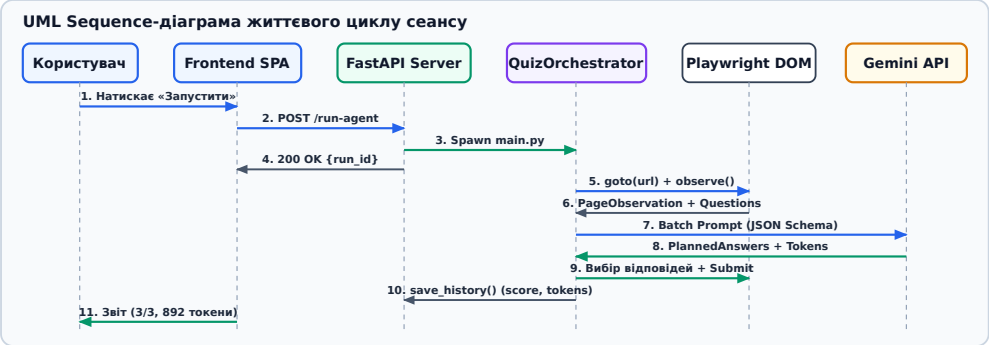


Рисунок 7.3. UML Sequence-діаграма взаємодії компонентів під час проходження тесту

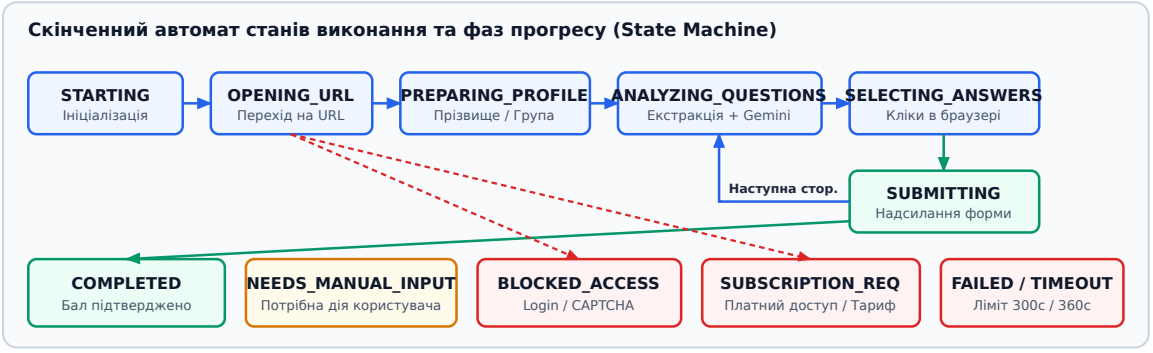


Рисунок 7.4. Скінченний автомат робочих фаз та термінальних статусів мультиагентного оркестратора

Розділ 8. Алгоритми інспекції DOM, планування відповідей та виконання дій

8.1. Чотирирівнева стратегія екстракції питань (`QuestionExtractor`)

Оскільки веб-платформи суттєво різняться за структурою HTML, агент `QuestionExtractor` (`src/browser_agent/agents/question_extractor.py`) застосовує каскад із чотирьох евристичних алгоритмів (див. **Рисунок 8.1**):

1. **Контейнерна екстракція (Structured Containers):** пошук семантичних блоків `fieldset`, `.question`, `[data-question]`, `[role="listitem"]`, де заголовок питання міститься у `legend` або `h2/h3/.question-title`, а варіанти — у дочірніх `label` або `input[type="radio"|"checkbox"]`.
2. **Групування за атрибутом `name` (Radio/Checkbox Name Grouping):** якщо явні контейнери відсутні, радіокнопки об’єднуються за спільним атрибутом `name`, а текст питання реконструюється з найближчого спільного предка у DOM-дереві.
3. **Аналіз ARIA-ролей та кастомних карток (ARIA & Card Heuristics):** розпізнавання сучасних SPA-компонентів із `role="radiogroup"`, `role="radio"`, `role="option"` або клікабельних блоків відповідей.
4. **Режим одного питання на екрані (Single-Question Screen Fallback):** для покрокових квізів, де весь основний контент сторінки становить одне запитання з набором кнопок-відповідей.

Перед передачею питань до планувальника виконується **фільтрація вже відповідених елементів**: якщо у групі радіокнопок або чекбоксів уже встановлено прапорець `checked === true` або `aria-checked="true"`, питання

позначається як виконане і не відправляється повторно до LLM.



Рисунок 8.1. Конвеєр мультистратегічної екстракції питань та пакетного планування відповідей через Google Gemini

8.2. Пакетне планування відповідей (`AnswerPlanner`) та облік токенів

Замість того щоб робити окремий мережевий виклик до Gemini для кожного питання чи кожного кліку, агент `AnswerPlanner` формує **єдиний структурований пакетний промпт** для всіх невідповідених питань поточного екрана. Модель повертає строго валідований JSON-масив об'єктів `PlannedAnswer` (ідентифікатор питання, обраний індекс/текст опції, коротка аргументація та рівень впевненості `confidence ∈ [0.0, 1.0]`). Сумарна витрата токенів за сеанс із K викликів моделі визначається за формулою:

$$T_{total} = \Sigma_{i=1..K} \left(T_{prompt}^{(i)} + T_{candidates}^{(i)} \right)$$

де $T_{prompt}^{(i)}$ — кількість вхідних токенів запиту (`usage_metadata.prompt_token_count`), а $T_{candidates}^{(i)}$ — кількість згенерованих токенів відповіді (`usage_metadata.candidates_token_count`). Як підтверджено на реальному запуску `af7e39f59aa04753af67e8d3fe753ea9` (див. Рисунок 5.5), проходження тесту з 3 питань вимагає лише `560 + 332 = 892` токени та займає 6 секунд.

8.3. Багаторівневе відновлення кліків (`BrowserActionExecutor`)

На реальних сайтах прямий виклик `locator.click()` часто блокується спливаючими cookie-банерами, фіксованими хедерами або прихованими нативними `◯`, стилізованими через сусідній ``. Модуль `action_executor.py` реалізує 5-кроковий алгоритм гарантованої дії з обов'язковою пост-верифікацією стану DOM (див. **Рисунок 8.2**).



Рисунок 8.2. Алгоритм виконання браузерних дій та багаторівневого відновлення при перехопленні кліків

Розділ 9. Моделі даних, атомарна персистентність та контроль життєвого циклу

9.1. Схеми даних Pydantic v2

Усі внутрішні та зовнішні структури даних типізовані за допомогою Pydantic v2 у модулях `src/browser_agent/models.py` та `src/browser_agent/schemas.py` (див. **Рисунок 9.1**). Це виключає помилки неузгодженості типів між оркестратором, FastAPI-сервером та клієнтським інтерфейсом.

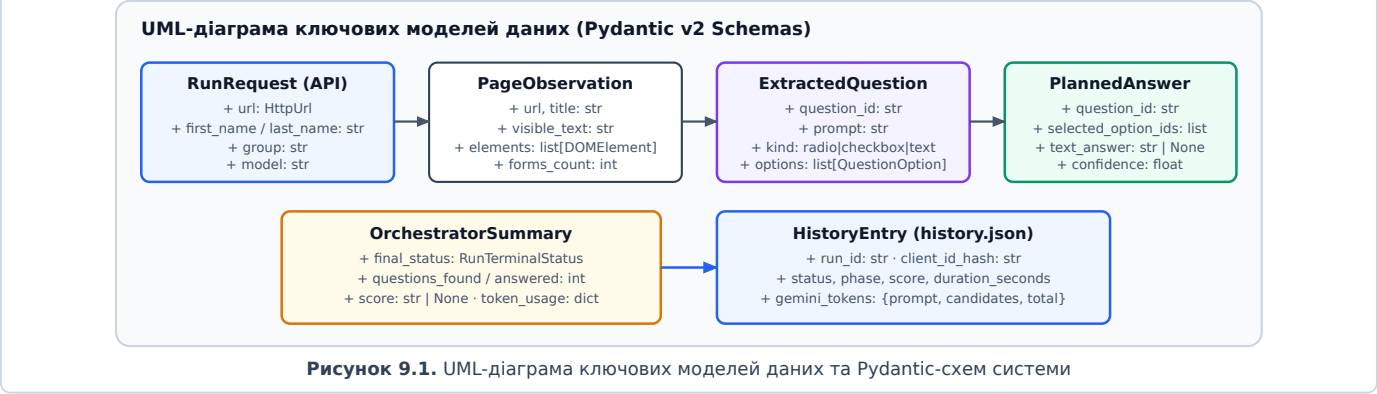


Рисунок 9.1. UML-діаграма ключових моделей даних та Pydantic-схем системи

9.2. Атомарне збереження історії, ізоляція пристроїв та захист від зависань

Для збереження історії сесій без використання важких зовнішніх СУБД реалізовано транзакційно-безпечний файловий механізм на базі `results/history.json` та індивідуальних лог-файлів `results/run_<run_id>.log` (див. **Рисунок 9.2**):

- **Атомарний запис через тимчасовий файл:** метод `save_history()` створює тимчасовий файл у тій самій файловій системі за допомогою `tempfile.mkstemp(dir="results", prefix="history_", suffix=".tmp")`, записує повний JSON із примусовим скиданням буфера та виконує атомарну системну операцію `os.replace(tmp_path, history_path)`. Навіть у разі раптового завершення контейнера файл `history.json` ніколи не залишається наполовину записаним або пошкодженим.
- **Криптографічна ізоляція історії пристроїв:** кожен клієнтський браузер передає свій UUID у заголовку `X-Client-Id`. Сервер обчислює `hashlib.sha256(client_id.encode()).hexdigest()[:16]` і зберігає хеш у полі `client_id` запису. При запиті `GET /history` повертаються виключно записи з відповідним хешем.
- **Подвійний захист від завислих запусків (Lifecycle & Stale-Run Watchdog):**
  1. Усередині самого агента діє жорсткий таймер виконання `MAX_RUN_DURATION_SECONDS = 300` (5 хвилин). При перевищенні ліміту підпроцес примусово закриває браузер і фіксує термінальний статус у `finally`-блоці.
  2. На рівні API-сервера функція `reconcile_stale_runs()` перевіряє всі записи зі статусом `running`: якщо процес уже завершився у таблиці процесів або триває понад `STALE_RUN_TIMEOUT_SECONDS = 360` (6 хвилин), статус автоматично переводиться у термінальний стан із діагностичним повідомленням.



Рисунок 9.2. Архітектура атомарного збереження історії, SHA-256 ізоляції пристроїв та реконсиляції завислих сесій

Розділ 10. Архітектура безпеки, захист від SSRF та межі довіри

10.1. Загроза Server-Side Request Forgery (SSRF) у хмарних браузерних агентах

Будь-який серверний додаток, який відкриває наданий користувачем URL у повноцінному браузері Chromium усередині хмарного середовища (Google Cloud Run), є критичною мішенню для SSRF-атак. Без належної фільтрації зловмисник міг би передати внутрішню адресу метаданих Google Cloud (`http://169.254.169.254/computeMetadata/v1/`), локальний сокет `http://127.0.0.1:8080` або вузли приватної підмережі VPC і викрасти сервісні токени через текст сторінки.

10.2. Багаторівневий захисний екран `public_url_security.py`

У коміті `4c081b6` (PR #7) реалізовано спеціалізований модуль мережевої безпеки `public_url_security.py`, який дозволяє користувачам проходити тести на будь-яких легітимних публічних освітніх платформах, одночасно блокуючи всі вектори SSRF (див. **Рисунок 10.1** та **Таблицю 10.1**).



Рисунок 10.1. Модель безпеки системи, межі довіри (Trust Boundaries) та фільтрація SSRF-загроз

Таблиця 10.1. Ешелонована система захисту периметра Browser Automation Agent

Рівень захисту	Механізм реалізації	Які загрози нейтралізує
1. Перевірка схеми та порту	Дозволено лише `https://` (стандартний порт `443`) для публічних сайтів; схеми `file://`, `javascript:`, `data:`, `ftp://` блокуються.	Локальне читання файлів контейнера (LFI) та виконання довільного JS-коду при старті.
2. Блокування внутрішніх доменів	Заборона хостів `localhost`, `metadata.google.internal` та суфіксів `.local`, `.internal`, `.localhost`, `.lan`.	Звернення до внутрішніх сервісів оркестрації Google Cloud та локальної мережі.
3. DNS-резолвінг та IP-аудит	Виклик `socket.getaddrinfo()` для отримання всіх IPv4/IPv6 адрес хоста та перевірка кожної адреси через стандартний модуль `ipaddress`.	Захист від DNS-спуфінгу на приватні діапазони: `127.0.0.0/8`, `10.0.0.0/8`, `172.16.0.0/12`, `192.168.0.0/16`, `169.254.0.0/16`, `::1`, `fc00::/7`.
4. Контроль редиректів у браузері	Повторна перевірка фінального URL у `browser.py` після кожного переходу `page.goto()` та під час навігації.	Нейтралізація Open Redirect / HTTP 302 перенаправлень із публічного домену на `169.254.169.254`.
5. Захист API та секретів	Перевірка `X-API-Key` через `hmac.compare_digest()`; зберігання `GEMINI_API_KEY` виключно на бекенді; заголовки HSTS та `nosniff`.	Несанкціоноване витрачання квот Gemini API, витік ключів у клієнтський JS та атаки типу Timing Attack.

Розділ 11. Хмарна інфраструктура розгортання та конвеєр CI/CD

11.1. Топологія розгортання у Vercel та Google Cloud Run

Продуктивне середовище розділене між двома спеціалізованими хмарними платформами (див. **Рисунок 11.1**):

- **Frontend на Vercel (`https://browser-agent-web-nine.vercel.app/`):** статичні ресурси роздаються через глобальну CDN-мережу Vercel з увімкненим `cleanUrls: true` у `vercel.json` та примусовим HTTPS (HSTS). Тут також доступні еталонні стенди `/test-quiz.html` та `/agent-test.html`.
- **Backend на Google Cloud Run (регіон `europe-west1`):** Docker-контейнер на базі `mcr.microsoft.com/playwright/python:v1.63.0-noble` запускає ASGI-сервер Uvicorn на порту `8080`.

> **Критична інфраструктурна вимога Cloud Run (CPU Always Allocated):** оскільки ендпоінт `POST /run-agent` повертає HTTP-відповідь миттєво, а підпроцес Playwright + Gemini продовжує виконуватися у фоні через `asyncio.create_task(collect_output(...))`, сервіс Cloud Run **обов'язково розгортається з прапорцем `--no-cpu-throttling`**. За стандартного режиму *request-based CPU* хмара заморожує процесор одразу після закриття HTTP-відповіді, що призводило до зависання підпроцесів.

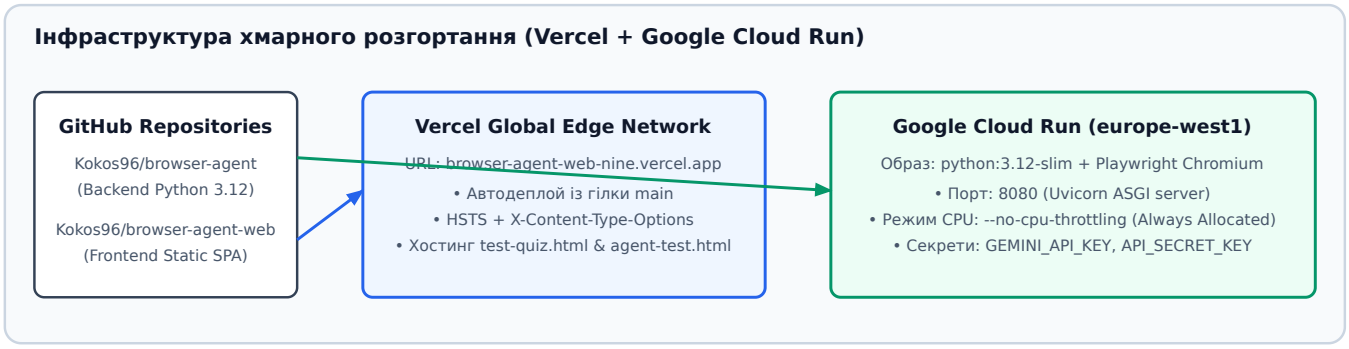


Рисунок 11.1. Архітектура хмарного розгортання клієнтської частини у Vercel та серверного контейнера у Google Cloud Run

```
# Еталонна команда розгортання Backend у Google Cloud Run (europe-west1)
gcloud run deploy browser-agent-api   --source .   --region europe-west1   --platform managed   --allow-unauthenticated   --
no-cpu-throttling   --memory 2Gi   --cpu 2   --timeout 360   --set-env-vars="GEMINI_MODEL=gemini-2.5-flash-
lite,MAX_RUN_DURATION_SECONDS=300,STALE_RUN_TIMEOUT_SECONDS=360"
```

11.2. Автоматизований конвеєр перевірки якості CI/CD

Кожен коміт і Pull Request у репозиторії `Kokos96/browser-agent` автоматично запускає GitHub Actions workflow `.github/workflows/validate.yml` на віртуальній машині `ubuntu-latest` з Python 3.12 (див. **Рисунок 11.2**). Конвеєр компілює всі вихідні файли через `python -m compileall`, встановлює браузер Chromium (`playwright install --with-deps chromium`) та проганяє повний набір із 65 модульних і браузерних інтеграційних тестів.

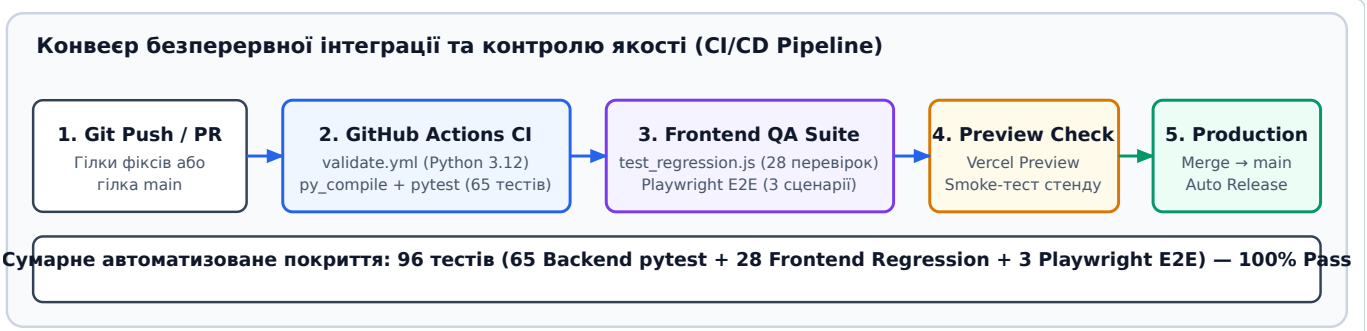


Рисунок 11.2. Конвеєр безперервної інтеграції (GitHub Actions CI), регресійного тестування та доставки у Production

Розділ 12. Стратегія тестування, продуктивність та аналіз ризиків

12.1. Багаторівнева піраміда тестування (96 автоматизованих перевірок)

Надійність системи підтверджена **96 автоматизованими тестами**, які охоплюють усі рівні — від чистих функцій безпеки до наскрізних браузерних сценаріїв (див. **Рисунок 12.1** та **Таблицю 12.1**). Крім того, до складу фронтенду включено два детерміновані контрольні стенди:

- `/test-quiz.html` **(«Контрольний стенд E2E»):** україномовний тест із 3 питань (*Сонце — це зоря; 4; H2O*) та полями профілю студента, який генерує фінальний бал `3/3`.
- `/agent-test.html` **(«Agent Smoke Test», коміт `7b5a632`):** мінімалістичний односторінковий стенд з одним питанням (вибір опції `PASS` із чотирьох радіокнопок `PASS / FAIL / MAYBE / SKIP`) та миттєвим виведенням результату `Your score: 1/1` у блок `#result[role="status"]`.

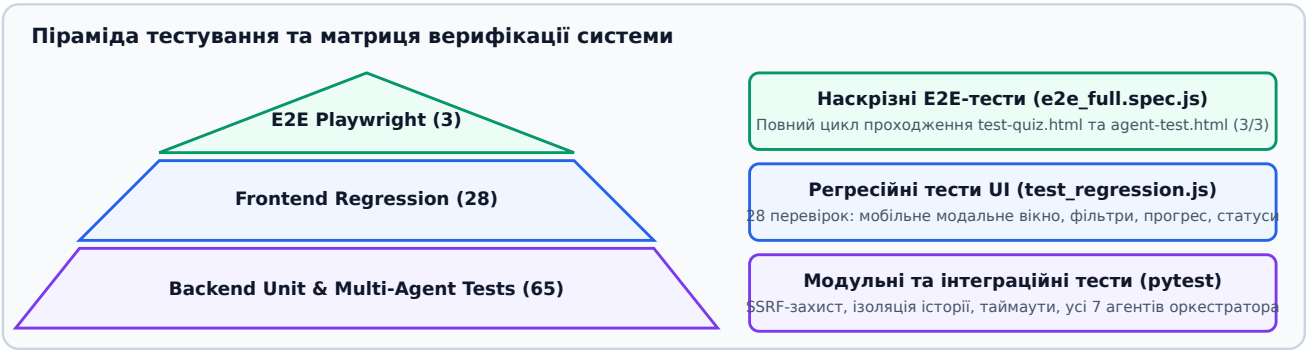


Рисунок 12.1. Піраміда автоматизованого тестування програмного комплексу (96 верифікованих тестів)

Таблиця 12.1. Зведений реєстр тестових наборів та результатів верифікації

Тестовий модуль	Репозиторій	К-сть	Що саме перевіряється	Результат
`tests/test_multi_agent_orchestrator.py`	`browser-agent`	33	Робота всіх 7 агентів, багатосторінкові квізи, фільтр відповідених питань, `SUBSCRIPTION_REQUIRED`, відновлення кліків, живий прогін у Chromium.	**33/33 PASS**
`tests/test_public_url_security.py`	`browser-agent`	13	Дозвіл публічних HTTPS URL, блокування `localhost`, приватних IP (RFC1918), `169.254.169.254`, небезпечних схем та редиректів.	**13/13 PASS**
`tests/test_run_timeout.py`	`browser-agent`	6	Таймаут життєвого циклу, збереження помилок в історії, атомарний `save_history()`, реконсиляція завислих сесій `reconcile_stale_runs()`.	**6/6 PASS**
`tests/test_history_isolation.py` + `test_agent_fixes.py`	`browser-agent`	13	Ізоляція історії між різними `X-Client-Id`, видалення записів, парсинг JSON-відповідей LLM, детекція полів профілю.	**13/13 PASS**
`test_regression.js`	`browser-agent-web`	28	Нормалізація об'єктних логів, мобільна верстка модального вікна, пресети `test-quiz.html` та `agent-test.html`, нові термінальні статуси.	**28/28 PASS**
`e2e_full.spec.js` (Playwright)	`browser-agent-web`	3	Повний наскрізний цикл у браузері: запуск зі сторінки SPA, моніторинг Live Activity, перевірка балу `3/3` та модального звіту в історії.	**3/3 PASS**

12.2. Профіль продуктивності, витрати токенів та матриця ризиків

Експериментальні вимірювання (див. **Рисунок 12.2**) підтверджують, що використання моделі `gemini-2.5-flash-lite` у поєднанні з пакетним плануванням відповідей забезпечує повний цикл проходження 3-питального тесту за **6–9 секунд** при витраті лише **~892 токенів**. Матрицю ключових технічних ризиків та механізмів їх нейтралізації узагальнено на **Рисунку 12.3**.



Рисунок 12.2. Порівняння швидкодії моделей Gemini та структура споживання токенів на 1 тестовий сеанс



Рисунок 12.3. Матриця інженерних ризиків експлуатації системи та впроваджених засобів захисту

Розділ 13. Перспективи розвитку проєкту та перехід на власну локальну LLM-інфраструктуру

13.1. Обмеження поточної хмарної моделі Google Gemini API

На поточному етапі розвитку система використовує зовнішній хмарний сервіс **Google Gemini API** як когнітивний рушій агента `AnswerPlanner` та резервного класифікатора сторінок. Попри простоту початкової інтеграції, експлуатація зовнішнього комерційного API створює чотири фундаментальні обмеження при масштабуванні проєкту:

1. **Квотні обмеження швидкості (Rate Limits — RPM / RPD / TPM):** безкоштовні та базові комерційні рівні Gemini API обмежують кількість запитів за хвилину (RPM) і на добу (RPD). При паралельному запуску десятків тестових сесій виникають помилки `429 RESOURCE_EXHAUSTED`.
2. **Пряма фінансова залежність від обсягу токенів:** при переході до масового регресійного тестування сотень багатосторінкових курсів або підключенні мультимодального аналізу скриншотів вартість токенів зростає лінійно.
3. **Конфіденційність та суверенітет даних (Data Privacy):** передача внутрішнього HTML/DOM-контенту корпоративних або університетських систем тестування на зовнішні сервери третіх сторін часто заборонена політиками інформаційної безпеки (NDA / GDPR).
4. **Неможливість глибокого донавчання ваг під специфіку DOM:** хмарна модель загального призначення не спеціалізована вузько під синтаксис DOM-дерев та українські освітні платформи (Moodle, НаУрок, Всеосвіта, Google Forms).

13.2. Архітектура переходу на власну локальну нейромережу (Self-Hosted Local LLM)

Завдяки тому, що вся взаємодія з мовною моделлю у бекенді вже інкапсульована в єдиному класі `LLMClient` (`src/browser_agent/llm.py`) і використовує строгі JSON-схеми Pydantic, система архітектурно готова до **повної заміни хмарного Gemini API на власний локальний або приватно розгорнутий LLM-кластер** (див. **Рисунок 13.1** та **Таблицю 13.1**).



Рисунок 13.1. Цільова архітектура v3.0: заміна хмарного Gemini API на власний локальний LLM-кластер (vLLM / Ollama + LoRA + Vision DOM)

Таблиця 13.1. Порівняльний аналіз хмарного Gemini API та власної локальної LLM-інфраструктури

Критерій порівняння	Поточна архітектура (Cloud Gemini API)	Цільова архітектура v3.0 (Власна локальна LLM)
Рушій інференсу (Inference Engine)	Хмарний Google GenAI REST/gRPC API	Локальний сервер **vLLM**, **Ollama** або **llama.cpp / TGI** з OpenAI-сумісним API (`/v1/chat/completions`)
Рекомендовані відкриті моделі	Пропрієтарні `gemini-2.5-flash-lite / flash`	**Qwen2.5-VL-7B/14B-Instruct** (мультимодальна), **Llama-3.3-70B-Instruct** (AWQ/FP8), **DeepSeek-R1-Distill-Qwen-14B**
Вартість токенів та ліміти квот	Тарифікація за кожен токен; жорсткі ліміти RPM/RPD	**0.00 $ за токен** (фіксована вартість власного GPU); **відсутність зовнішніх квот RPM/RPD**
Приватність даних (Data Sovereignty)	DOM-зрізи передаються у зовнішню хмару Google	**100% локальна обробка** у межах власного контуру без виходу даних у зовнішній Інтернет
Контроль формату виводу (Structured Output)	Хмарна JSON-генерація	Детерміноване граматичне декодування (**Guided Decoding / Outlines / XGrammar** у vLLM) — 100% валідний JSON за схемою Pydantic
Спеціалізація під предметну область	Обмежена лише системним промптом (In-Context)	Можливість **QLoRA / LoRA донавчання** на тисячах реальних DOM-знімків українських та міжнародних тестових платформ

13.3. Технічні кроки реалізації локальної нейромережі та масштабування продукту

1. **Впровадження мультипровайдерного адаптера у `llm.py`:** розширення конфігурації змінною `LLM_PROVIDER=local|gemini` та `LOCAL_LLM_BASE_URL=http://localhost:8000/v1`. При виборі локального режиму запити спрямовуються до сервера `vLLM` або `Ollama` із увімкненим параметром `response_format={"type": "json_schema", "json_schema": PlannedAnswersBatch.model_json_schema()}`.

2. **Мультимодальний візуальний аналіз сторінки (Vision-Language DOM Fusion):** підключення локальної візуально-мовної моделі (наприклад, `Qwen2.5-VL-14B`), яка одночасно отримує очищене DOM-дерево та скриншот поточного вікна Playwright (`page.screenshot()`). Це дозволить агенту розв’язувати питання з математичними формулами на зображеннях, графіками, кресленнями та Canvas-діаграмами.
3. **Донавчання власного адаптера (LoRA Fine-Tuning):** накопичені логи успішних запусків із `results/history.json` формують готовий датасет пар *«DOM-структура питання → верифікована дія та правильна відповідь»*. Донавчання компактної 7B/14B моделі за методом QLoRA на одній відеокарті NVIDIA RTX 4090 / L4 (24 ГБ VRAM) забезпечить точність рівня великих пропрієтарних моделей при швидкості генерації `> 90 токенів/с`.
4. **Трансформація у корпоративну AI-QA платформу:** заміна постійного файлу `history.json` на PostgreSQL / Cloud Firestore для мультикористувацьких команд, заміна клієнтського поллінгу на **Server-Sent Events (SSE) / WebSockets** з прямою трансляцією кадрів браузера в UI та створення розширення Chrome (Manifest V3) для запуску агента безпосередньо у вже авторизованій сесії користувача.

Додатки А–Б. Довідник діагностичних вердиктів та Глосарій

Таблиця Д.1. Довідник термінальних статусів сеансу та їх інтерпретація у Frontend SPA

Статус API (`status`)	Бейдж в інтерфейсі	Умова виникнення в `QuizOrchestrator`	Рекомендована дія користувача
`completed`	**Завершено** (зелений)	Вибрано відповіді, форму надіслано та підтверджено фінальний бал або екран результату.	Переглянути розпізнаний бал та звіт у модальному вікні історії.
`needs_manual_input`	**Потрібна дія** (жовтий)	Відповіді вибрано, але кнопка сабміту заблокована або потребує ручного підтвердження.	Перевірити форму або завершити надсилання вручну.
`blocked_access`	**Доступ обмежено** (помаранчевий)	Детектовано сторінку входу (Login/OAuth), закритий корпоративний тест або CAPTCHA.	Використовувати публічно доступне посилання на квіз без обов’язкового входу.
`subscription_required`	**Потрібна підписка** (помаранчевий)	Виявлено платний тарифний екран (Paywall / Pro-підписка) замість тестових питань.	Перевірити доступність безкоштовного режиму на цільовій платформі.
`error` / `failed`	**Помилка** (червоний)	Перевищено ліміт `300 с`, помилка мережі або не знайдено жодного питання на сторінці.	Переглянути секцію «Діагностика» та повні логи сесії у деталях запуску.

Підсумковий інженерний висновок: Програмний комплекс **Browser Automation Agent v2.5** є повністю завершеним, захищеним від SSRF та верифікованим 96 автотестами продуктом із сучасною 7-агентною архітектурою. Закладена модульна структура дозволяє без переписування бізнес-логіки оркестратора перевести систему з хмарного Gemini API на власну локальну нейромережу, перетворивши її на автономну високопродуктивну платформу інтелектуальної веб-автоматизації.